

CS203 Spring 2026 - Final Project - Load Value Prediction

Deadline: Friday, June 5 @ 23:59 Pacific Time.

Note: Slip days cannot be used for the final project.

Project logistics

Group project / Submission

The final project can be conducted in **groups of up to 3 students**. You can choose to have a smaller group of 2 or as an individual.

Once you have formed your group join the following Github Classroom assignment and create a repo for your group: https://classroom.github.com/a/mmpLyb_P

The repository that will be created is empty since this is an open-ended project. You will use this repository to submit your final project with the deliverables below.

Deliverables

- Report that describes:
 - Your approach towards converging to a load value predictor design (Why did you design it that way?)
 - The design and implementation of your predictor(s) (How did you design it?)
 - Your experimental setup/methodology (How did you test your design?)
 - Evaluation and analysis of your predictor (Does your design work?)
- Documentation / instructions on how to compile and run your predictor.
 - Include the exact commands that was used to generate your results and to evaluate your predictor
- Source code
- Sample traces which you used to evaluate and test See the generating memory load traces section below.

Load value prediction overview

The purpose of load value prediction is to be able to *predict* the *value* of a *memory load* instruction.

Recall that memory accesses can be hundreds or thousands of cycles and can lead to significant pipeline stalls, especially on cache misses.

One unique solution around this is to be able to predict the value of the load instruction and *speculatively execute* based on the *predicted* load value.

Later when the *actual* memory load value returns, we can compare and check to see if our load value was correctly predicted or not.

Similar to our branch misprediction scenario in speculative tomasulo, if our load value is *mispredicted* we simply *rollback* by flushing the ROB.

If we correctly predicted, then great! We continue running on and benefited from executing instructions speculatively.

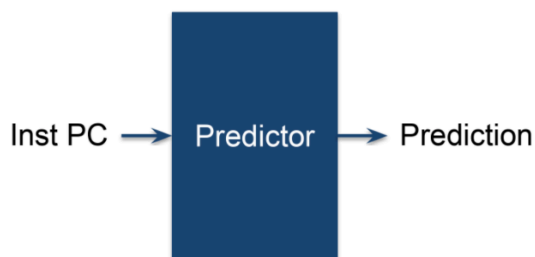
A major reason why value prediction works is that programs tend to exhibit *value similarity* behaviors where values tend to be similar either spatially (neighboring memory addresses) or temporally (repeated access to the same memory location does not change, for example, a constant value in memory).

If you're curious, you can refer to the following seminal paper by Lipasti, et. al for details: [Value Locality and Load Value Prediction](#)

You can abstractly think of value prediction as the figure below:

Value Prediction [Lipasti&al.96][Mendelson&al.97]

▪ Instruction-based VP:



▪ Prediction functions:

- **Last Value:** Pred = Result of last instance.
- **Stride:** Pred = Result of last instance + constant.
- **FCM:** Pred based on results of n recent instances.
- **VTAGE:** $\approx$ Control-flow based Last Value Predictor.



Given an instruction PC (which represents "What memory load instruction is this?"), a predictor function is able to predict the value that will be read from memory.

There are various examples of simple predictors, such as using the last value that was loaded from this address, the last value + a stride, using global load value information, or using control-flow information. These predictor functions have the same limitations as our branch predictor, in that it is resource limited and typically based on tables of information.

In addition to the **predicted load value**, load value predictors also include **confidence values**, that is, "Should I use the predicted value of this load instruction or not?"

The load value predictor typically learns if a certain load instruction has load value properties that can be extracted or not.

For example, if you have random numbers populated in memory, then a load value predictor should not be used since you'll probably mispredict a lot and end up rolling back a lot which will make your processor slower overall.

Final project requirements

Your goal for the final project is to be able to develop a load value predictor.

Load value predictor requirements

The predictor will use a memory access trace that contains the load PC, the load address and the load value as detailed in the next section. The inputs to the predictor will be:

- The PC address of the load instruction
- The memory address to be access by the load

The output of the predictor will be:

- The predicted value
- Whether you should use the predicted value or not (the confidence). For simplicity, this output should be binary. (Yes/No)

Load value predictor implementation

There are two options for this final project.

Option 1: Practical load value predictor -- Similar to our class assignment, you can create a table-based predictor that will try to achieve as high of an accuracy as possible. This is completely open-ended so you can choose to try different designs to improve accuracy. For comparison, you are *required* to implement a simple last value predictor where you simply predict the value that was last loaded for that load PC and use the prediction for every load.

Option 2: Magical machine learning load value predictor -- Go crazy and have fun. Imagine your load value predictor can have infinite resources and compute power and can run a whole ML framework in it. Use Tensorflow, Keras, pyTorch, insert ML framework here, to build your predictor. This is open-ended so you can design any type of ML that you find suitable. Note that most likely it will be some kind of reinforcement learning since you have to learn as the program is executing (as you're going through the trace). The goal here is not to perform supervised learning (off-line training using a subset of the trace as a training set and another subset as testing set) since load value predictors in real hardware cannot be pre-trained and must learn on-the-fly.

Generating memory load traces with Pin Dynamic Binary Instrumentation tool

For reference, Pin Manual: <https://software.intel.com/sites/landingpage/pintool/docs/98547/Pin/html/>

Download and compile

- Download Pin 3.22 (Linux, gcc): [pin-3.22-98547-g7a303a835-gcc-linux.tar.gz](https://software.intel.com/sites/landingpage/pintool/docs/98547/Pin/html/#download)
- Extract the tar.gz file
- Go into the extracted directory: `cd pin-3.22*/source/tools/ManualExamples`
- Compile the tool: `make all TARGET=intel64`

Run the pintool to collect a memory trace

Below is an example command to run the pin tool on the `ls` command.

```
../../pin -t obj-intel64/pinatrace.so -- /bin/ls
```

Memory trace is output to file `pinatrace.out` and contains the PC address, read/write to memory, and the memory address.

```
$ head pinatrace.out
0x7f2498c8f093: W 0x7ffc630ad988
0x7f2498c8fe90: W 0x7ffc630ad980
0x7f2498c8fe94: W 0x7ffc630ad978
0x7f2498c8fe96: W 0x7ffc630ad970
0x7f2498c8fe98: W 0x7ffc630ad968
0x7f2498c8fe9a: W 0x7ffc630ad960
0x7f2498c8fe9f: W 0x7ffc630ad958
0x7f2498c8feaf: R 0x7f2498cb5e78
0x7f2498c8feb6: W 0x7f2498cb56e0
0x7f2498c8fec7: R 0x7f2498cb6000
```

Modify `pinatrace.cpp` to print out load data values only

Since we care only about load value prediction, we can **comment out** lines 42–46 of `pinatrace.cpp` (the `if (INS_MemoryOperandIsWritten...)` block inside `Instruction()`), which instruments store instructions.

Modify the `RecordMemRead()` function on line 16 to print out the value loaded from memory. Replace the one-line version that comes with Pin 3.22 with the expanded version below:

```
// Print a memory read record
VOID RecordMemRead(VOID * ip, VOID * addr)
{
    ADDRINT value;
    PIN_SafeCopy(&value, addr, sizeof(ADDRINT));
    fprintf(trace,"%p: R %p, 0x%016lx\n", ip, addr,value);
}
```

Now you can collect load value traces. (Make sure you recompile.)

Example: Make a trace of your assignment 1 program

Run your pintool on your assignment 1 pipesim similar to below. You will likely have to replace the path that is suitable for your environment.

```
pin-3.22-98547-g7a303a835-gcc-linux/source/tools/ManualExamples$ ../../../../pin -t obj-intel64/pinatrace.so -- ~/CS
```

The memory load value traces will be in `pinatrace.out`. For my pipesim, the output file is 29MB, containing 604K of memory loads with 209K unique memory loads (note there are many duplicate memory load values!)

```
pin-3.22-98547-g7a303a835-gcc-linux/source/tools/ManualExamples$ ls -lh pinatrace.out
-rw-r--r-- 1 dwong dwong 29M Feb 21 14:10 pinatrace.out
pin-3.22-98547-g7a303a835-gcc-linux/source/tools/ManualExamples$ wc -l pinatrace.out
604177 pinatrace.out
pin-3.22-98547-g7a303a835-gcc-linux/source/tools/ManualExamples$ sort pinatrace.out | uniq | wc -l
208910
```



For your final project, you will need to generate different load value traces to train/test your load value predictor



No Canvas quiz for the final project. Your grade is based on the deliverables listed above.